

Práctica Guiada: Listas Dinámicas y CRUD Local con Provider en Flutter

🎯 Objetivo general

Crear una aplicación Flutter que permita **agregar, editar y eliminar usuarios** utilizando el patrón **MVVM con Provider**, reforzando el manejo del estado, la UI dinámica y el trabajo con formularios.

🎯 Objetivos específicos

- Aplicar el patrón MVVM para separar la lógica de negocio y la interfaz.
 - Implementar **Provider** para manejar el estado de forma eficiente.
 - Usar componentes interactivos como **RadioButton** y **Switch**.
 - Comprender cómo funcionan las listas dinámicas en Flutter (**ListView.builder**).
-

🧠 Introducción teórica

MVVM (Model-View-ViewModel) es una arquitectura que separa la **vista** (UI) de la **lógica** (ViewModel), facilitando la reutilización del código y el mantenimiento de la aplicación.

Provider es una librería oficial recomendada por Flutter para manejar el estado. Permite que la vista se actualice automáticamente cuando el ViewModel notifica un cambio, usando **ChangeNotifier**.

En esta práctica, construiremos una app que gestiona una lista de usuarios en memoria (sin base de datos), mostrando cómo manejar formularios, listas dinámicas y actualizaciones reactivas.

Estructura del proyecto

lib/

|

|— main.dart

|— models/

| |— user.dart

|— viewmodels/

| |— user_view_model.dart

|— views/

| |— user_list_screen.dart

| |— user_form_screen.dart

NOTA: Añadir dependencia provider en pubspec.yaml:

dependencies:

flutter:

sdk: flutter

provider: ^6.1.5+1

Ejecutar en terminal: **flutter pub get** ó **directo en android studio.**

Desarrollo paso a paso

❶ Crear el modelo **User**

 **models/user.dart**

```
class User {  
  String nombre;  
  
  String genero; // 'Masculino' o 'Femenino'  
  
  bool activo;  
  
  User({  
    required this.nombre,  
    required this.genero,  
    required this.activo,  
  });  
}
```

❷ Crear el ViewModel **UserViewModel**

 **viewmodels/user_view_model.dart**

```
import 'package:flutter/material.dart';  
  
import '../models/user.dart';  
  
class UserViewModel extends ChangeNotifier {
```

```
final List<User> _usuarios = [];  
  
List<User> get usuarios => _usuarios;  
  
void agregarUsuario(User usuario) {  
    _usuarios.add(usuario);  
    notifyListeners();  
}  
  
void editarUsuario(int index, User usuario) {  
    _usuarios[index] = usuario;  
    notifyListeners();  
}  
  
void eliminarUsuario(int index) {  
    _usuarios.removeAt(index);  
    notifyListeners();  
}  
}
```

3 Configurar **Provider** en el **main.dart**

 **main.dart**

```
import 'package:flutter/material.dart';
```

```
import 'package:provider/provider.dart';
import 'viewmodels/user_view_model.dart';
import 'views/user_list_screen.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => UserViewModel(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Gestión de Usuarios',
      theme: ThemeData(primarySwatch: Colors.indigo),
      home: const UserListScreen(),
    );
  }
}
```

```
}
```

4 Crear la pantalla principal **UserListScreen**

 **views/user_list_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../viewmodels/user_view_model.dart';
import '../models/user.dart';
import 'user_form_screen.dart';

class UserListScreen extends StatelessWidget {
  const UserListScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final viewModel = context.watch<UserViewModel>();

    return Scaffold(
      appBar: AppBar(title: const Text('Lista de Usuarios')),
      body: ListView.builder(
        itemCount: viewModel.usuarios.length,
        itemBuilder: (context, index) {
          final user = viewModel.usuarios[index];
```

```

        return Card(
            child: ListTile(
                title: Text(user.nombre),
                subtitle: Text('${user.genero} - ${user.activo ?
'Activo' : 'Inactivo}')),
                trailing: Row(
                    mainAxisAlignment: MainAxisAlignment.min,
                    children: [
                        IconButton(
                            icon: const Icon(Icons.edit),
                            onPressed: () async {
                                final actualizado = await Navigator.push(
                                    context,
                                    MaterialPageRoute(
                                        builder: (_) => UserFormScreen(
                                            usuario: user,
                                            indice: index,
                                        ),
                                    ),
                                );
                                if (actualizado != null && actualizado is
User) {
                                    viewModel.editarUsuario(index,
actualizado);
                                }
                            }
                        )
                    ]
                )
            )
        );
    }
}

```

```

        },
    ),
    IconButton(
        icon: const Icon(Icons.delete, color:
Colors.red),
        onPressed: () =>
viewModel.eliminarUsuario(index),
    ),
],
),
),
);
},
),
floatingActionButton: FloatingActionButton(
    onPressed: () async {
        final nuevoUsuario = await Navigator.push(
            context,
            MaterialPageRoute(builder: (_) => const
UserFormScreen()),
        );
        if (nuevoUsuario != null && nuevoUsuario is User) {
            viewModel.agregarUsuario(nuevoUsuario);
        }
    },
),

```



```
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

5 Crear el formulario **UserFormScreen**

 **views/user_form_screen.dart**

```
import 'package:flutter/material.dart';
import '../models/user.dart';

class UserFormScreen extends StatefulWidget {
  final User? usuario;
  final int? indice;

  const UserFormScreen({super.key, this.usuario, this.indice});

  @override
  State<UserFormScreen> createState() => _UserFormScreenState();
}

class _UserFormScreenState extends State<UserFormScreen> {
  final _formKey = GlobalKey<FormState>();
```

```
late String _nombre;

String _genero = 'Masculino';

bool _activo = true;

@override
void initState() {
  super.initState();

  if (widget.usuario != null) {
    _nombre = widget.usuario!.nombre;
    _genero = widget.usuario!.genero;
    _activo = widget.usuario!.activo;
  } else {
    _nombre = '';
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text(widget.usuario == null ? 'Agregar Usuario' :
'Editar Usuario'),
    ),
    body: Padding(
      padding: const EdgeInsets.all(16.0),
```

```

child: Form(
  key: _formKey,
  child: Column(
    children: [
      TextFormField(
        initialValue: _nombre,
        decoration: const InputDecoration(labelText:
'Nombre'),
        validator: (value) =>
          value == null || value.isEmpty ? 'Ingrese un
nombre válido' : null,
        onSave: (value) => _nombre = value!,
      ),
      const SizedBox(height: 20),
      const Text('Género'),
      Row(
        children: [
          Expanded(
            child: RadioListTile(
              title: const Text('Masculino'),
              value: 'Masculino',
              groupValue: _genero,
              onChanged: (value) => setState(() =>
_genero = value!),
            ),

```

```

    ),
    Expanded(
      child: RadioListTile(
        title: const Text('Femenino'),
        value: 'Femenino',
        groupValue: _genero,
        onChanged: (value) => setState(() =>
_genero = value!)),
    ),
  ),
],
),
SwitchListTile(
  title: const Text('Activo'),
  value: _activo,
  onChanged: (value) => setState(() => _activo =
value),
),
const SizedBox(height: 30),
ElevatedButton(
  onPressed: () {
    if (_formKey.currentState!.validate()) {
      _formKey.currentState!.save();

      final user = User(nombre: _nombre, genero:
_genero, activo: _activo);

```

```
        Navigator.pop(context, user);
    }
},
child: Text(widget.usuario == null ? 'Guardar' :
'Actualizar'),
),
],
),
),
),
);
}
```

Preguntas de reflexión

1. ¿Qué ventajas ofrece usar **Provider** frente a **setState()**?

Provider permite separar lógica y UI (mejor arquitectura).

Actualiza solo los widgets necesarios (no toda la pantalla).

Facilita compartir estado entre pantallas.

Es más escalable y ordenado en proyectos medianos/grandes.

Reduce errores al centralizar el estado.

2. ¿Por qué es importante usar **ChangeNotifier** en el ViewModel?

Permite que el ViewModel notifique cambios a la UI.

Hace que Provider pueda reconstruir widgets automáticamente.

Habilita el uso de `notifyListeners()`.

Convierte al ViewModel en un gestor de estado reactivo.

3. ¿Qué sucedería si no se llamara a `notifyListeners()` después de editar un usuario?

La interfaz no se actualiza.

Los cambios existen en memoria, pero no se reflejan en pantalla.

Se rompe la reactividad del patrón MVVM.

El usuario cree que nada cambió.

Actividades extras/complementarias

1. Agregar un campo de edad (TextField numérico) y mostrarlo en la lista.
 2. Validar que el nombre no esté vacío y que la edad sea mayor a 0.
 3. Agregar un campo “Correo electrónico” con validación de formato.
 4. Implementar un filtro para mostrar solo usuarios activos.
-

Conclusión

Durante esta práctica, se implementó un CRUD local aplicando el patrón MVVM con Provider.

Los estudiantes comprendieron cómo separar la lógica de negocio de la interfaz, manipular listas dinámicas, utilizar componentes interactivos y validar formularios.

Este enfoque promueve buenas prácticas en el desarrollo móvil y sienta las bases para integrar almacenamiento local o remoto en futuras clases.